Sebenta 07

Automatização de Configurações - IaC

Infraestruturas de Computação na Nuvem | Aula 07

Objeto e utilização deste documento

O presente documento acompanha a apresentação da Aula 07. Cada página reproduz um slide e desenvolve, em texto corrido, a matéria nele condensada. A aula é expositiva e prolonga as anteriores: a abordagem declarativa já conhecida do Kubernetes (Aula 05), a ideia de que a configuração de rede pode ser centralizada e programada (Aula 06) e o isolamento e a imutabilidade das imagens de containers (Aula 04).

A leitura sequencial é a mais proveitosa. A aula parte da tensão entre desenvolvimento e operações que originou o movimento DevOps e das limitações da configuração manual de servidores. Apresenta depois o princípio de Infrastructure as Code e as duas famílias de ferramentas que o concretizam, a gestão de configuração com o Ansible e o provisionamento com o Terraform. Termina com a gestão de segredos, o CI/CD aplicado a infraestrutura e uma síntese de conceitos.

Automatização de Configurações - IaC

Infraestruturas de Computação na Nuvem, Aula 07

Catarina Silva

ESTGA, Universidade de Aveiro | 2026

A Aula 06 mostrou que a rede de um datacenter pode ser gerida como software, a partir de um ponto central. A pergunta que esta aula coloca é mais geral: o que acontece quando toda a infraestrutura, e não apenas a rede, passa a ser descrita e aplicada como código?

A resposta é a automatização da configuração e do provisionamento, conhecida pela expressão Infrastructure as Code (IaC). A aula apresenta as motivações que a justificam, os princípios que a sustentam e as ferramentas que a tornam prática, em particular o Ansible e o Terraform.

Objetivos de Aprendizagem

- Justificar a automatização da configuração e do provisionamento face à gestão manual de infraestrutura.
- Distinguir abordagens declarativas e imperativas e explicar o conceito de idempotência.
- Descrever a arquitetura e os componentes do Ansible.
- Explicar o funcionamento do Terraform, em particular o state file e o ciclo plan/apply.
- Diferenciar provisionamento de gestão de configuração.
- Aplicar boas práticas de gestão de segredos e CI/CD a infraestrutura.

Os objetivos de aprendizagem organizam o estudo em três eixos. O primeiro é conceptual: justificar a automatização face à gestão manual e distinguir as abordagens declarativa e imperativa, com o conceito de idempotência. O segundo é técnico: descrever o Ansible, explicar o Terraform e o seu state file e diferenciar provisionamento de gestão de configuração.

O terceiro eixo é de prática profissional: aplicar boas práticas de gestão de segredos e de CI/CD à infraestrutura. Ao terminar o estudo deste documento, o estudante deve conseguir explicar cada um destes pontos com as suas próprias palavras e identificar, perante um cenário concreto, qual a ferramenta adequada.

Agenda (1/2)

- DevOps e a tensão entre desenvolvimento e operações.
- Limitações da configuração manual.
- Infrastructure as Code (IaC).
- Abordagens declarativa e imperativa; idempotência.
- Gestão de configuração com Ansible.

A primeira metade da agenda trata da motivação e dos fundamentos. Começa pelo DevOps, entendido como resposta organizacional à tensão entre quem desenvolve software e quem opera a infraestrutura, e pelas limitações da configuração manual, que tornam essa tensão dispendiosa.

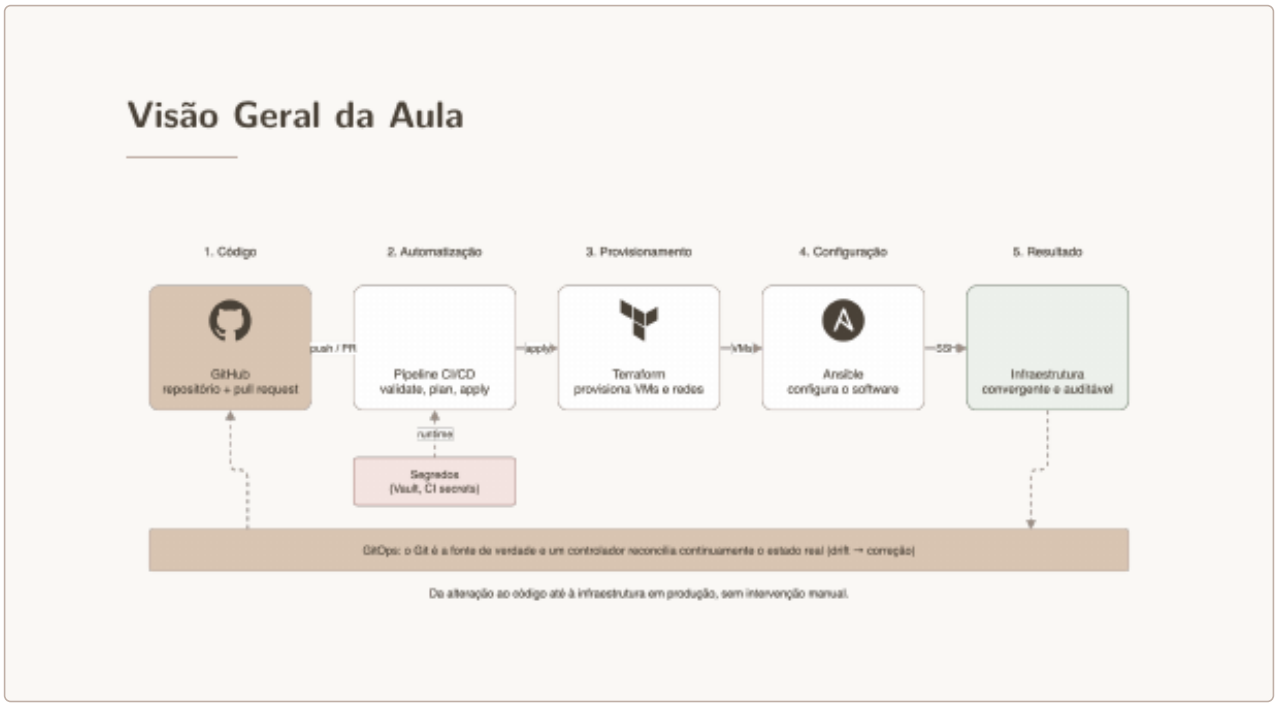
Seguem-se o princípio de Infrastructure as Code, a distinção entre as abordagens declarativa e imperativa, com a noção de idempotência, e a gestão de configuração com o Ansible.

Agenda (2/2)

- Provisionamento declarativo com Terraform e gestão do state file.
- Provisionamento versus gestão de configuração.
- Gestão de segredos.
- CI/CD aplicado a infraestrutura.

A segunda metade da agenda passa do conceito à prática. Apresenta o provisionamento declarativo com o Terraform e o papel do state file, e esclarece a diferença entre provisionamento e gestão de configuração, uma das confusões mais frequentes neste tema.

Termina com dois assuntos transversais a qualquer ferramenta: a gestão de segredos, que impede que credenciais acabem expostas no repositório, e o CI/CD aplicado a infraestrutura, que automatiza a validação e a aplicação das alterações.



O diagrama resume o percurso da aula como um fluxo único. Uma alteração nasce num repositório Git, no GitHub, sob a forma de commit e de pull request. Um pipeline de CI/CD valida o código, gera um plano e, após revisão, aplica-o. O Terraform provisiona as máquinas e as redes, e o Ansible configura o software que corre nelas, resultando numa infraestrutura convergente e auditável.

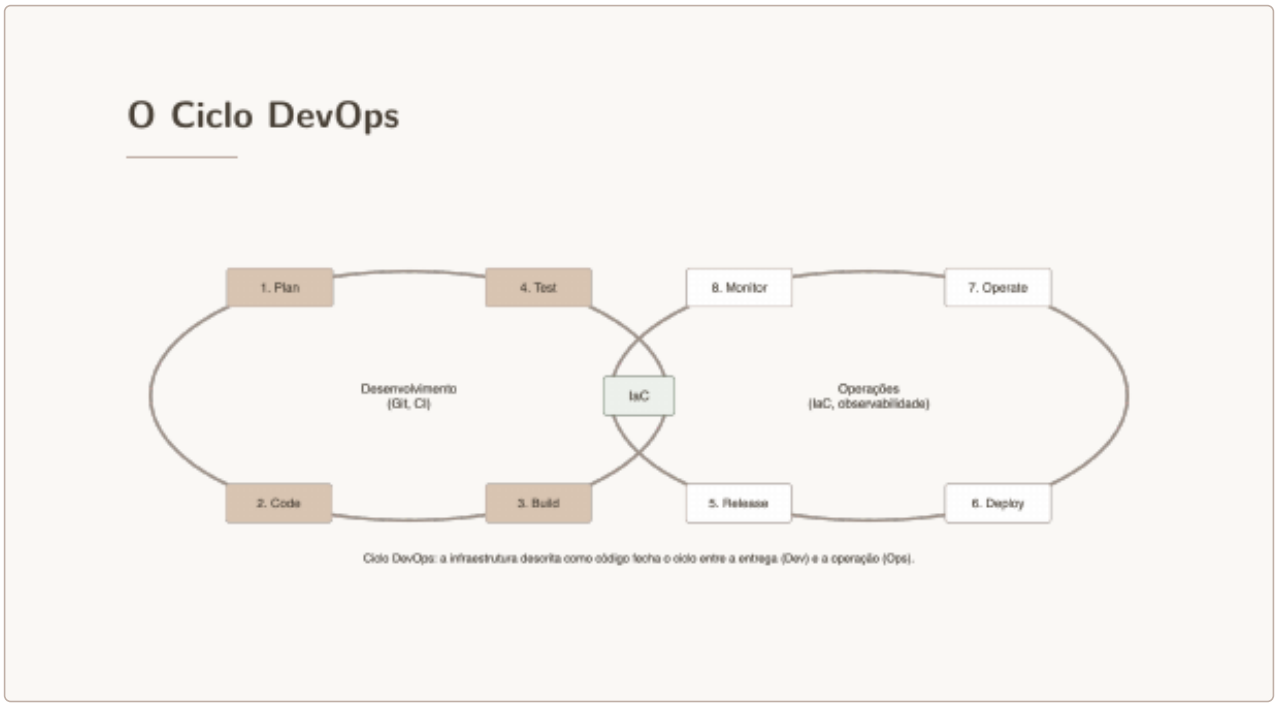
Dois elementos acompanham o fluxo. Os segredos são fornecidos ao pipeline apenas em tempo de execução, e não ficam no repositório. A barra inferior representa a reconciliação contínua, em que o Git funciona como fonte de verdade e qualquer divergência do estado real é detetada e corrigida. Cada bloco será desenvolvido nas páginas seguintes.

DevOps e a Tensão Entre Desenvolvimento e Operações

- As equipas de **Dev** (evolução do software) e de **Ops** (estabilidade da infraestrutura) têm incentivos historicamente opostos: maximizar a frequência de alterações versus minimizar o risco de falha.
- A separação organizacional gera handoffs manuais, ambientes inconsistentes e lead times elevados entre commit e produção.
- A resposta burocrática (aprovações manuais, janelas de alteração rígidas) aumenta o lead time sem melhorar a qualidade.
- **DevOps** designa a cultura e o conjunto de práticas que integram as duas funções num único ciclo, assente em versionamento, integração e entrega contínuas, e automatização.

Historicamente, as equipas de desenvolvimento (Dev) e de operações (Ops) têm incentivos opostos. As primeiras são avaliadas pela rapidez com que entregam funcionalidades, as segundas pela estabilidade dos sistemas em produção, e cada alteração é, para quem opera, um risco. Esta separação gera passagens de testemunho manuais, ambientes inconsistentes e um lead time elevado entre o commit e a entrada em produção.

A resposta burocrática, feita de aprovações manuais e janelas de alteração rígidas, agrava o problema: aumenta o lead time sem melhorar a qualidade. O DevOps surge como a cultura e o conjunto de práticas que integram as duas funções num único ciclo, apoiado em versionamento, integração e entrega contínuas e automatização.



O símbolo do infinito representa o ciclo DevOps como um processo contínuo, e não como uma sequência com princípio e fim. O laço do lado do desenvolvimento reúne o planeamento, a escrita do código, a construção (build) e os testes. O laço do lado das operações reúne a disponibilização (release), a implantação (deploy), a operação e a monitorização.

A monitorização alimenta o planeamento seguinte, fechando o ciclo. No cruzamento dos dois laços está a infraestrutura descrita como código, porque é ela que garante que o ambiente onde o software é testado e o ambiente onde é operado são o mesmo, e que ambos podem ser recriados a pedido.

Cenários de Aplicação do DevOps

- Na integração contínua, cada commit desencadeia a compilação e a execução automática de testes.
- Na entrega e implantação contínuas, as versões seguem para produção de forma frequente e reversível.
- As implantações progressivas (canary, blue-green) e as feature flags limitam o impacto de cada alteração.
- A criação de ambientes efémeros de teste por pull request exige infraestrutura provisionada a pedido.
- A observabilidade (métricas, logs, alertas) alimenta o planeamento do ciclo seguinte.
- O DevSecOps integra no pipeline a análise de dependências e a deteção de segredos expostos.

O DevOps materializa-se em cenários concretos de engenharia de software. Na integração contínua, cada commit desencadeia a compilação e a execução automática de testes. Na entrega e implantação contínuas, as versões seguem para produção de forma frequente e reversível.

As implantações progressivas, como canary e blue-green, e as feature flags limitam o impacto de cada alteração, porque expõem a nova versão a uma fração dos utilizadores ou permitem desligá-la sem nova implantação. A criação de ambientes efémeros por pull request exige infraestrutura provisionada a pedido. A observabilidade, com métricas, logs e alertas, alimenta o planeamento seguinte, e o DevSecOps integra no pipeline a análise de dependências e a deteção de segredos expostos.

Relevância para a Unidade Curricular

O DevOps surge nesta unidade curricular porque a automatização da infraestrutura é a condição técnica que permite aos cenários anteriores serem executados de forma contínua, repetível e auditável.

Este slide justifica a presença do tema nesta unidade curricular. Todos os cenários anteriores pressupõem que os ambientes de destino existem, estão atualizados e são equivalentes entre si. Quando a infraestrutura é configurada manualmente, torna-se o elo não automatizado do ciclo e reintroduz as passagens manuais que o DevOps procura eliminar.

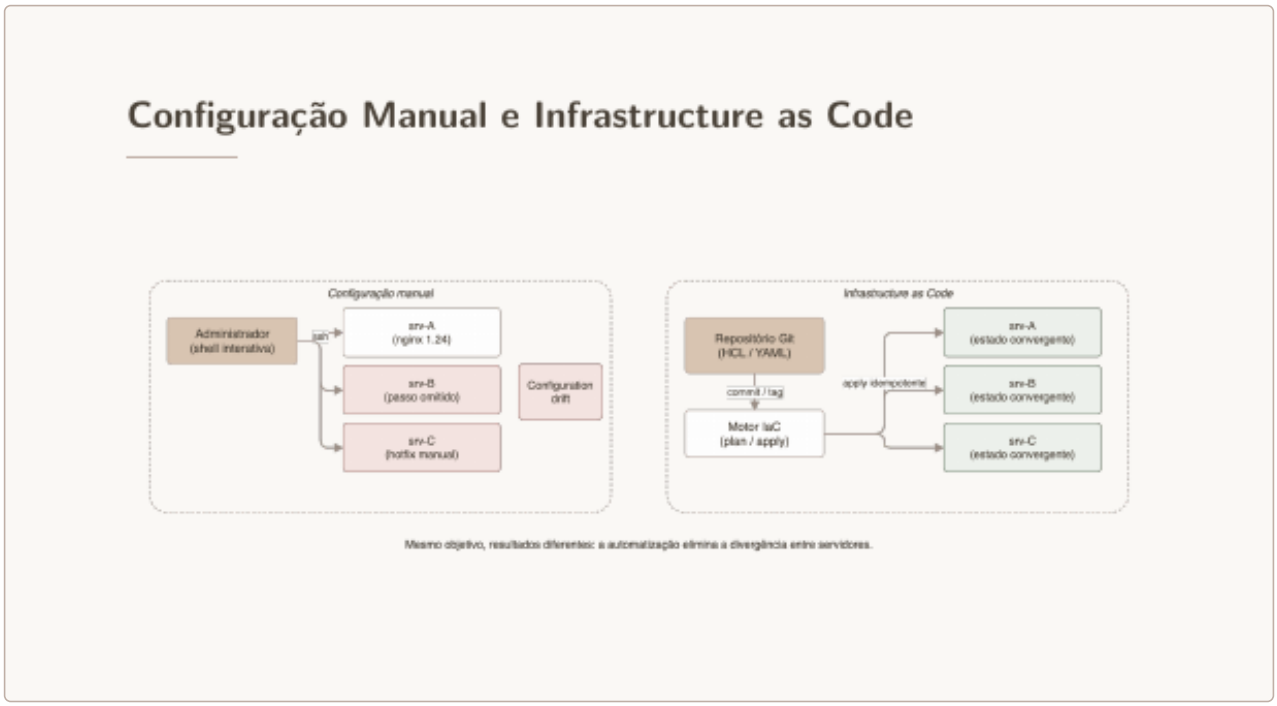
A automatização da infraestrutura é, por isso, a condição técnica para que esses cenários possam ser executados de forma contínua, repetível e auditável. É nesse sentido que o DevOps é aqui abordado.

Limitações da Configuração Manual

- Não escala: o esforço cresce linearmente com o número de servidores.
- O **configuration drift** resulta da evolução independente de cada servidor e é difícil de detetar sem inventário do estado real.
- O erro humano (passos omitidos, comandos mal aplicados) gera inconsistências difíceis de diagnosticar.
- Os ambientes de desenvolvimento, teste e produção divergem, pelo que os testes deixam de ser representativos.
- A recuperação de desastres é lenta, pois recriar a infraestrutura manualmente demora horas ou dias (RTO elevado).
- A auditoria e a conformidade exigem registo de quem alterou o quê e quando.

A configuração manual de servidores tem limitações estruturais. Não escala, porque o esforço cresce linearmente com o número de máquinas. Origina *configuration drift*, a evolução independente de cada servidor, difícil de detetar sem um inventário do estado real. E expõe-se ao erro humano, com passos omitidos ou comandos mal aplicados que geram inconsistências difíceis de diagnosticar.

Há ainda consequências operacionais. Os ambientes de desenvolvimento, teste e produção divergem, pelo que os testes deixam de ser representativos. A recuperação de desastres é lenta, porque recriar a infraestrutura à mão demora horas ou dias, o que se traduz num *recovery time objective* (RTO) elevado. Por fim, a auditoria e a conformidade exigem um registo de quem alterou o quê e quando, que a prática manual raramente fornece.



O diagrama compara as duas abordagens. À esquerda, um administrador liga-se por SSH a cada servidor e aplica as alterações à mão. Um servidor fica com um passo omitido, outro recebe um *hotfix* manual, e o conjunto diverge ao longo do tempo sem que haja registo do que mudou.

À direita, a definição desejada vive num repositório Git, em ficheiros HCL ou YAML. Um motor de IaC lê essa definição e aplica-a de forma idempotente a todos os servidores, que convergem para o mesmo estado. A diferença não está no objetivo, que é o mesmo, mas no processo: o primeiro depende da memória e da disciplina de quem executa, o segundo é repetível por construção.

Infrastructure as Code (IaC)

- A infraestrutura (servidores, redes, serviços) é descrita em ficheiros de texto (YAML, HCL) e não através de passos manuais.
- O código é versionado em Git, o que permite histórico, revisão por pares (code review) e rollback para qualquer versão.
- A **idempotência** garante que aplicar a mesma definição várias vezes converge sempre para o mesmo estado.
- A **repetibilidade** permite gerar ambientes idênticos em qualquer momento e testar alterações antes de produção (lint, dry-run).
- O código constitui documentação atualizada e reduz o tempo de provisionamento de dias para minutos.

Infrastructure as Code consiste em descrever a infraestrutura, isto é, servidores, redes e serviços, em ficheiros de texto, em vez de a construir através de passos manuais. Esses ficheiros, escritos em linguagens como YAML ou HCL, são versionados em Git, o que dá histórico, revisão por pares e a possibilidade de repor qualquer versão anterior.

Duas propriedades tornam a abordagem útil. A idempotência garante que aplicar a mesma definição várias vezes converge sempre para o mesmo estado. A repetibilidade permite gerar ambientes idênticos em qualquer momento e testar alterações antes de chegarem a produção, com verificações de sintaxe (*lint*) e execuções simuladas (*dry-run*). O código funciona ainda como documentação atualizada e reduz o tempo de provisionamento de dias para minutos.

Do Princípio às Ferramentas

- O IaC é um princípio e não uma ferramenta: define que a infraestrutura deve ser descrita, versionada e aplicada como código.
- A sua concretização envolve dois problemas distintos.
- O primeiro consiste em garantir que os recursos existem (VMs, redes, storage), o que corresponde ao **provisionamento**, abordado pelo **Terraform**.
- O segundo consiste em definir o que corre dentro de cada recurso (pacotes, ficheiros, serviços), o que corresponde à **gestão de configuração**, abordada pelo **Ansible**.
- Em ambos os casos aplicam-se os mesmos fundamentos, a descrição declarativa do estado e a idempotência.

O IaC é um princípio, e não uma ferramenta. Estabelece que a infraestrutura deve ser descrita, versionada e aplicada como código, mas não diz como. A sua concretização envolve dois problemas distintos, e cada um tem as suas ferramentas.

O primeiro é garantir que os recursos existem, como máquinas virtuais, redes e armazenamento. É o provisionamento, de que o Terraform é o exemplo estudado. O segundo é definir o que corre dentro de cada recurso, nomeadamente pacotes, ficheiros de configuração e serviços. É a gestão de configuração, de que o Ansible é o exemplo estudado. Em ambos se aplicam os mesmos fundamentos, a descrição declarativa do estado e a idempotência.



O Ansible é uma ferramenta open source de gestão de configuração e automatização. Aplica, por SSH e sem necessitar de um agente instalado, o estado descrito em ficheiros YAML denominados playbooks a máquinas que já existem.

O Terraform é uma ferramenta open source de provisionamento declarativo. A partir de código HCL, cria, altera e destrói recursos de infraestrutura, como máquinas virtuais, redes e armazenamento, recorrendo às APIs dos *providers*, que são os plugins específicos de cada plataforma. As duas ferramentas respondem, portanto, a perguntas diferentes: o Terraform decide que recursos existem, o Ansible decide o que neles corre.

Abordagens Imperativa e Declarativa

- Na abordagem **imperativa** descreve-se a sequência de passos (por exemplo, um script shell), pelo que o resultado depende do estado inicial.
- Na abordagem **declarativa** descreve-se o estado final e a ferramenta calcula as ações necessárias, tal como no Kubernetes (Aula 05).
- Esta abordagem torna possível a **idempotência**: reaplicar a definição converge para o mesmo estado, e as ferramentas reportam o que mudou (*changed*) e o que já estava conforme (*ok*).

Na abordagem imperativa descreve-se a sequência de passos a executar, como acontece num script shell que instala um pacote, copia um ficheiro e reinicia um serviço. O resultado depende do estado inicial da máquina, e repetir o script pode produzir efeitos indesejados, como acrescentar duas vezes a mesma linha a um ficheiro.

Na abordagem declarativa descreve-se o estado final pretendido, e é a ferramenta que calcula as ações necessárias para lá chegar. É o mesmo princípio do Kubernetes, estudado na Aula 05. Esta abordagem torna possível a idempotência, segundo a qual reaplicar a definição converge sempre para o mesmo estado. As ferramentas distinguem o que mudou (*changed*) do que já estava conforme (*ok*), o que torna cada execução auditável.

Ansible

- Comunica por SSH (porta 22) e não exige agente: cada módulo é transferido para o host, executado com Python e removido.
- Os playbooks são escritos em **YAML** e descrevem o estado desejado de forma declarativa.
- O **inventory** lista os hosts geridos, agrupados por função.
- Os módulos (`apt`, `copy`, `service`) são idempotentes.
- O Ansible é mantido pela Red Hat e distribuído como software open source.

O Ansible comunica por SSH, na porta 22, e não exige agente nos hosts geridos. Para cada task, o módulo correspondente é transferido para o host, executado com Python e removido no final. O requisito mínimo no destino é, por isso, um servidor SSH e um interpretador Python.

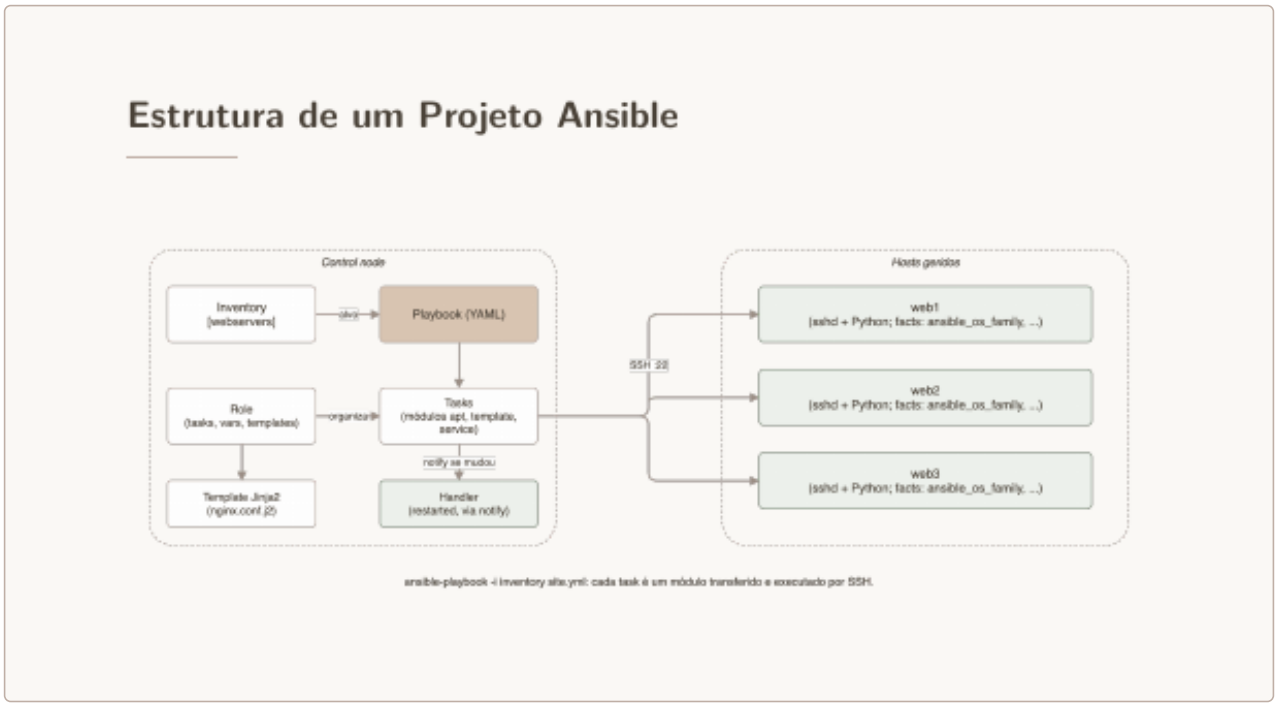
Os playbooks são escritos em YAML e descrevem o estado desejado de forma declarativa. O *inventory* lista os hosts geridos, agrupados por função, por exemplo num grupo de servidores web. Os módulos, como `apt`, `copy` e `service`, são idempotentes, pelo que um playbook pode ser executado repetidamente. O Ansible é mantido pela Red Hat e distribuído como software open source.

Componentes de um Projeto Ansible

- Um **playbook** é um ficheiro YAML que associa um conjunto de tasks a um grupo de hosts.
- Uma **task** é uma ação individual, executada por um **módulo**.
- Um **handler** é uma task executada apenas quando outra notifica que algo mudou (por exemplo, reiniciar o serviço após alterar a configuração).
- Uma **role** organiza tasks, variáveis, templates e ficheiros numa unidade reutilizável.
- Um **template Jinja2** é um ficheiro de configuração parametrizado por variáveis, instanciado para cada host.
- Os **facts** são informação recolhida automaticamente sobre cada host (sistema operativo, memória, endereços IP).

Um projeto Ansible organiza-se em torno de alguns componentes. Um playbook é um ficheiro YAML que associa um conjunto de tasks a um grupo de hosts. Cada task é uma ação individual, executada por um módulo.

Um handler é uma task especial, executada apenas quando outra notifica que algo mudou, por exemplo para reiniciar um serviço depois de alterar o seu ficheiro de configuração. Uma role agrupa tasks, variáveis, templates e ficheiros numa unidade reutilizável. Um template Jinja2 é um ficheiro de configuração parametrizado por variáveis, instanciado para cada host. Os *facts* são informação recolhida automaticamente sobre cada host, como o sistema operativo, a memória e os endereços IP, e podem ser usados em condições e templates.



O diagrama mostra como estes componentes se relacionam. No control node, o comando `ansible-playbook` lê o inventory, que define os hosts alvo, e o playbook, que define o que aplicar. O playbook usa tasks, que invocam módulos, e as tasks podem ser organizadas em roles, que incluem templates Jinja2.

Quando uma task altera algo, notifica o handler associado, que é executado no fim, e apenas uma vez. As tasks são enviadas por SSH para os hosts geridos, onde são executadas e onde são recolhidos os facts. O control node mantém todo o estado desejado, e os hosts apenas executam.

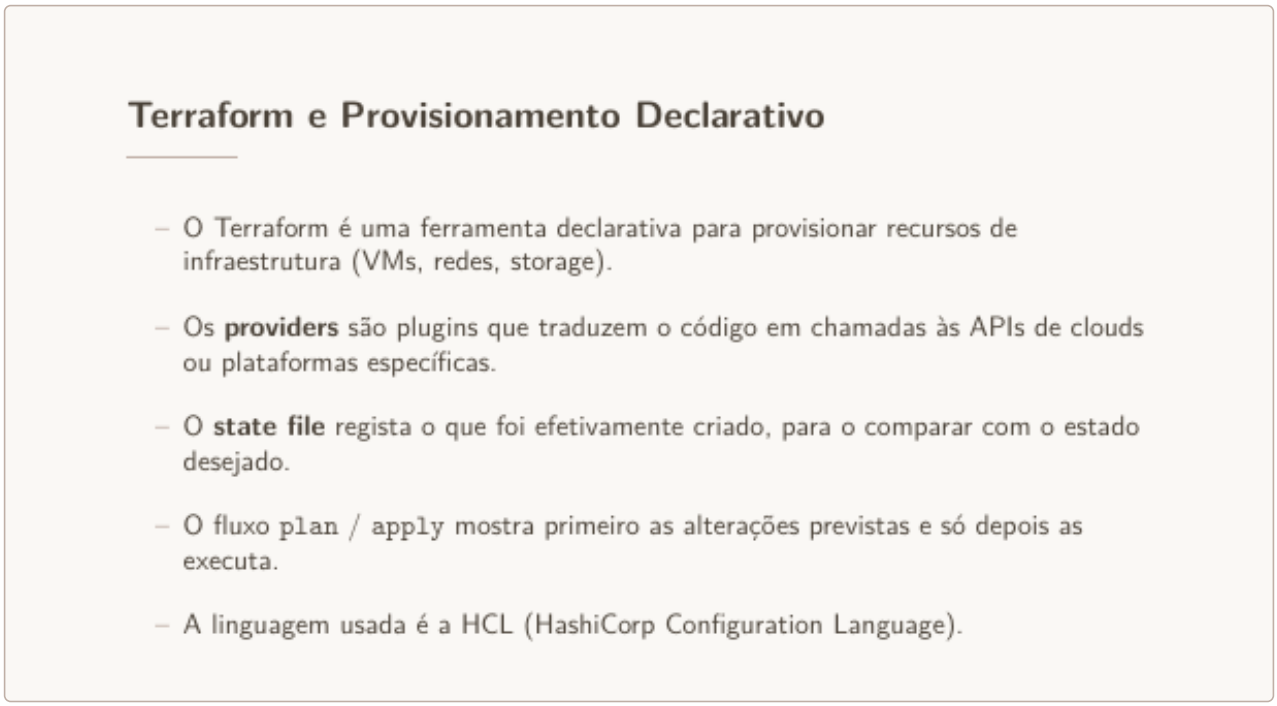
Exemplo de Playbook

```
- hosts: webservers
  become: true
  tasks:
    - name: Instalar o Nginx
      ansible.builtin.apt:
        name: nginx
        state: present
    - name: Copiar a configuração
      ansible.builtin.template:
        src: nginx.conf.j2
        dest: /etc/nginx/nginx.conf
      notify: Reiniciar o Nginx
  handlers:
    - name: Reiniciar o Nginx
      ansible.builtin.service:
        name: nginx
        state: restarted
```

Numa segunda execução sem alterações, todas as tasks reportam *ok* e o handler não é executado.

O exemplo mostra um playbook mínimo para servidores web. A primeira task usa o módulo `apt` para garantir que o pacote Nginx está instalado (`state: present`). A segunda copia o ficheiro de configuração a partir de um template Jinja2 e, se o conteúdo mudar, notifica o handler.

O handler reinicia o serviço, mas só quando notificado. Numa segunda execução sem alterações, todas as tasks reportam *ok* e o handler não é executado. Este comportamento ilustra a idempotência em prática: o playbook descreve o estado, e não os passos, e só atua quando há diferença.



O Terraform é uma ferramenta declarativa para provisionar recursos de infraestrutura. Os *providers* são plugins que traduzem o código em chamadas às APIs de clouds ou de plataformas específicas, como o Proxmox. O state file regista o que foi efetivamente criado, para que possa ser comparado com o estado desejado.

O fluxo de trabalho assenta nos comandos `plan` e `apply`. O primeiro mostra as alterações previstas e o segundo executa-as, o que permite rever o impacto antes de o aplicar. A linguagem usada é a HCL, *HashiCorp Configuration Language*, desenhada para descrever recursos de forma legível.

Exemplo de Configuração Terraform

```
terraform {
  required_providers {
    proxmox = { source = "bpg/proxmox" }
  }
}

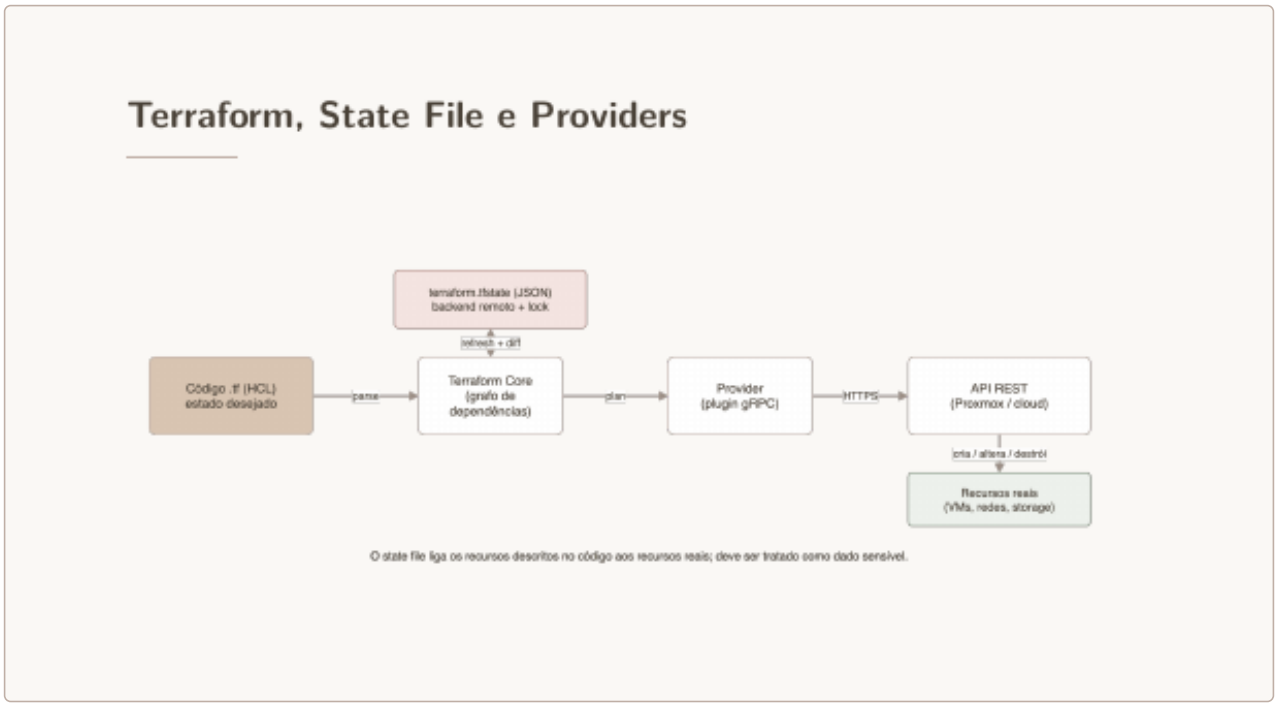
variable "vm_count" { default = 2 }

resource "proxmox_virtual_environment_vm" "web" {
  count     = var.vm_count
  name      = "web-${count.index + 1}"
  node_name = "pve1"
  cpu    { cores = 2 }
  memory { dedicated = 2048 }
}
```

Alterar `vm_count` para 3 e executar `terraform plan` indica a criação de uma única VM adicional.

O exemplo declara dois blocos principais. O bloco `terraform` indica o provider necessário, neste caso o do Proxmox. O bloco `resource` descreve o recurso a criar, uma máquina virtual com dois núcleos e 2048 MB de memória, cujo nome é calculado a partir de um índice.

A variável `vm_count` define quantas máquinas existem. Alterar o seu valor de 2 para 3 e executar `terraform plan` mostra a criação de uma única máquina adicional, porque o Terraform compara o estado desejado com o que já existe e calcula apenas a diferença. Não é necessário descrever como criar a máquina nem escrever lógica para evitar duplicados.



O diagrama mostra a arquitetura interna do Terraform. O código `.tf` é lido pelo núcleo (*core*), que constrói um grafo de dependências entre recursos. O núcleo consulta e atualiza o state file, que reside num backend remoto com bloqueio, e determina o plano de ações.

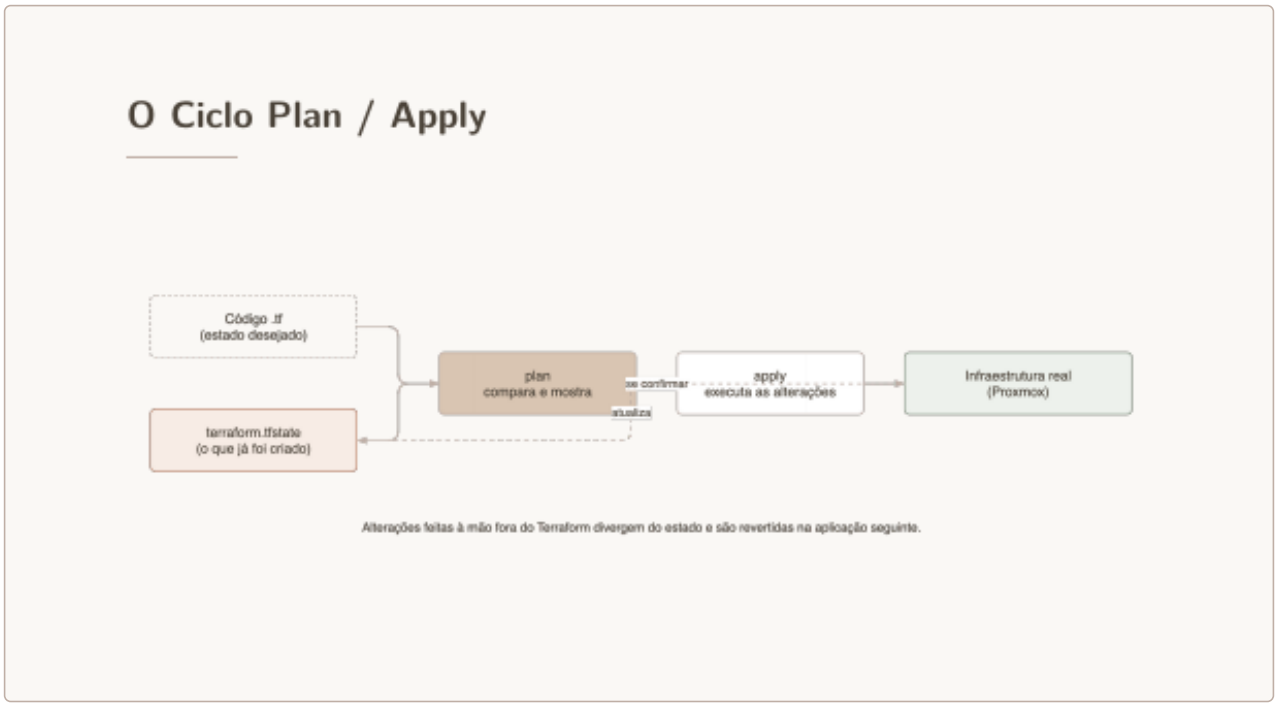
Esse plano é entregue ao provider, um plugin que comunica com o núcleo por gRPC, e que por sua vez chama a API REST da plataforma, por HTTPS, para criar, alterar ou destruir os recursos reais. O state file liga, assim, os recursos descritos no código aos recursos reais, e por isso deve ser tratado como um dado sensível.

O State File

- O **state file** regista o mapeamento entre os recursos descritos no código e os recursos reais.
- Permite ao Terraform determinar, em cada execução, o que já existe, o que falta criar e o que deve ser destruído.
- Em equipa, o estado deve residir num **backend remoto** partilhado, com **locking** para evitar execuções simultâneas.
- O ficheiro pode conter dados sensíveis em texto simples e deve ser tratado como um segredo.
- Alterações manuais fora do Terraform introduzem drift e serão revertidas na aplicação seguinte.

O state file regista o mapeamento entre os recursos descritos no código e os recursos reais. É este mapeamento que permite ao Terraform determinar, em cada execução, o que já existe, o que falta criar e o que deve ser destruído.

Em trabalho de equipa, o estado deve residir num backend remoto partilhado, com *locking* para impedir execuções simultâneas que o corrompam. Como pode conter dados sensíveis em texto simples, deve ser tratado como um segredo. Alterações manuais feitas fora do Terraform introduzem drift e serão revertidas na aplicação seguinte, o que reforça a regra de que toda a alteração deve passar pelo código.



O diagrama mostra o ciclo *plan* e *apply*. O comando `plan` lê o código, que descreve o estado desejado, e o state file, que descreve o que já foi criado, compara-os e mostra as alterações previstas. Nada é modificado nesta fase.

Se o operador confirmar, o `apply` executa as alterações na infraestrutura real, neste caso o Proxmox, e atualiza o state file. A separação em duas fases é uma salvaguarda importante. Permite rever, e num pipeline de CI/CD aprovar, o que vai acontecer antes de acontecer.

Provisionamento e Gestão de Configuração

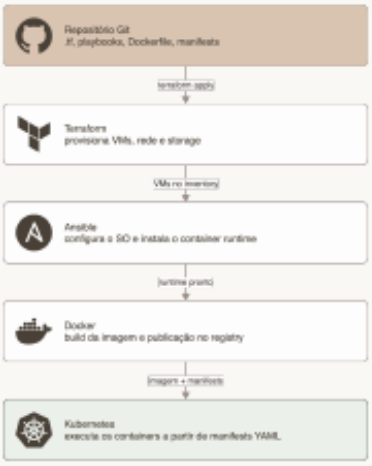
- O **provisionamento** decide que máquinas e recursos existem (por exemplo, Terraform).
- A **gestão de configuração** decide o que corre dentro de cada máquina (por exemplo, Ansible, Puppet).
- Fluxo habitual: o Terraform cria a VM e o Ansible instala e configura o software.
- As duas camadas são complementares e não concorrentes.

Este slide fixa a distinção entre as duas camadas. O provisionamento decide que máquinas e recursos existem, e o Terraform é o exemplo. A gestão de configuração decide o que corre dentro de cada máquina, e o Ansible, tal como o Puppet, são exemplos.

O fluxo habitual combina as duas: o Terraform cria a máquina virtual e o Ansible instala e configura o software. As camadas são complementares e não concorrentes, e confundi-las é a origem mais comum de dúvidas sobre qual ferramenta usar para uma determinada tarefa.

Integração com Docker, Kubernetes, Terraform e Ansible

- O Terraform provisiona VMs, rede e storage, incluindo os nós do cluster.
- O Ansible configura esses nós e instala o container runtime.
- O Docker empacota a aplicação numa imagem imutável, publicada num registry pelo CI/CD.
- O Kubernetes executa a imagem de forma declarativa, a partir de manifests YAML.
- Tudo é versionado no Git e aplicado pelo mesmo pipeline.



Esta página mostra como as tecnologias estudadas ao longo da unidade curricular se encadeiam. O Terraform provisiona as máquinas, a rede e o armazenamento, incluindo os nós de um cluster. O Ansible configura esses nós e instala o container runtime. O Docker empacota a aplicação numa imagem imutável, publicada num registry pelo pipeline de CI/CD. O Kubernetes executa essa imagem de forma declarativa, a partir de manifests YAML.

A ordem apresentada é uma simplificação, porque a construção da imagem não depende de o Ansible ter corrido. O essencial é que cada camada é descrita em código, versionada no Git e aplicada pelo mesmo pipeline, o que garante consistência entre a infraestrutura de base e a aplicação.

Gestão de Segredos

- O código de infraestrutura é versionado em Git, mas passwords, chaves e tokens não podem ser guardados no repositório em texto simples.
- O **Ansible Vault** cifra ficheiros de variáveis, permitindo guardá-los no repositório.
- O **HashiCorp Vault** e serviços equivalentes armazenam segredos de forma centralizada e fornecem-nos às ferramentas apenas em tempo de execução.
- As variáveis de ambiente podem ser injetadas pelo sistema de CI/CD, sem nunca serem escritas no repositório.
- Um segredo que tenha sido alguma vez submetido (commit) deve ser considerado comprometido e rodado.

O código de infraestrutura é versionado em Git, mas passwords, chaves e tokens não podem ser guardados no repositório em texto simples. Este é o principal desafio de segurança do IaC.

O Ansible Vault cifra ficheiros de variáveis, o que permite guardá-los no repositório. O HashiCorp Vault e serviços equivalentes armazenam os segredos de forma centralizada e fornecem-nos às ferramentas apenas em tempo de execução. As variáveis de ambiente podem ser injetadas pelo sistema de CI/CD, sem nunca serem escritas no repositório. Um segredo que tenha sido submetido (*commit*) deve ser considerado comprometido e rodado, porque permanece no histórico do Git mesmo depois de apagado do ficheiro.

CI/CD para Infraestrutura

- Aplicam-se ao código de infraestrutura os mesmos princípios de integração e entrega contínuas usados no software.
- O pipeline valida a sintaxe, gera o plano (`plan` ou dry-run), submete-o a revisão e só então executa o `apply`.
- Os testes automatizados antecedem qualquer alteração em produção, o que reduz o risco de alterações manuais não controladas.



Nenhuma alteração chega a produção sem validação, plano revisto e aprovação.

Aplicam-se ao código de infraestrutura os mesmos princípios de integração e entrega contínuas usados no software. O pipeline valida a sintaxe, gera o plano, submete-o a revisão e só então executa o *apply*.

O diagrama mostra estas etapas em sequência: o *push* ou pull request inicia o pipeline, seguem-se as validações de formato e sintaxe, a geração do plano como artefacto, a revisão por pares e, por fim, a aplicação num runner de CI. Se qualquer etapa falhar, o pipeline termina com um código de saída diferente de zero e a alteração não avança. Os testes automatizados antecedem qualquer alteração em produção, o que reduz o risco de alterações manuais não controladas.

Síntese de Conceitos de Infraestrutura

Conceito	Definição técnica	Exemplo
Idempotência	f(f(x)) = f(x): reaplicar a definição não altera o estado	`state: present`
Configuration drift	Divergência entre o estado real e o declarado	Edição manual de `nginx.conf`
Declarativo	Especifica-se o estado final; o motor calcula o delta	Playbook, ficheiro `.tf`
State file	Mapeamento entre recursos declarados e reais	`terraform.tfstate`
Agentless / agent-based	Push por SSH versus pull por agente residente	Ansible; Puppet, Chef
Imutabilidade	Substituição da instância em vez de alteração in-place	Imagem Packer / Docker
Reconciliação	Ciclo contínuo observar, comparar e corrigir	Argo CD, Flux

A tabela reúne os conceitos técnicos da aula. A idempotência significa que reaplicar a definição não altera o estado. O *configuration drift* é a divergência entre o estado real e o declarado, por exemplo uma edição manual de um ficheiro de configuração. Na abordagem declarativa especifica-se o estado final e o motor calcula a diferença.

O state file é o mapeamento entre recursos declarados e reais. A distinção entre agentless e agent-based opõe o modelo de *push* por SSH, do Ansible, ao de *pull* por um agente residente, do Puppet e do Chef. A imutabilidade substitui a instância em vez de a alterar no local, como acontece com imagens Packer ou Docker. A reconciliação é o ciclo contínuo de observar, comparar e corrigir, que ferramentas como o Argo CD e o Flux aplicam.

Síntese da Aula (1/2)

- O DevOps responde à tensão entre Dev e Ops através de cultura, automatização e versionamento.
- A configuração manual conduz a drift, erros e falta de reprodutibilidade.
- O IaC trata a infraestrutura como código versionado, idempotente e repetível.
- O Ansible (agentless, SSH, YAML) contrasta com Puppet e Chef (agent-based), que mantêm um agente em cada host.

A primeira parte da síntese recolhe os fundamentos. O DevOps responde à tensão entre desenvolvimento e operações através de cultura, automatização e versionamento. A configuração manual conduz a drift, erros e falta de reprodutibilidade.

O IaC trata a infraestrutura como código versionado, idempotente e repetível. O Ansible, agentless e baseado em SSH e YAML, contrasta com o Puppet e o Chef, que mantêm um agente em cada host e verificam periodicamente o estado, uma opção adequada a parques de grande dimensão.

Síntese da Aula (2/2)

- O Terraform provisiona recursos de forma declarativa (providers, state, plan/apply) e o state file é crítico e sensível.
- Provisionamento e gestão de configuração são camadas complementares.
- Os segredos nunca vão para o repositório em texto simples.
- O CI/CD automatiza o teste e a aplicação de alterações, com o Git como fonte de verdade.

A segunda parte da síntese recolhe as ferramentas e as práticas. O Terraform provisiona recursos de forma declarativa, com providers, state e o ciclo plan e apply, e o state file é crítico e sensível. O provisionamento e a gestão de configuração são camadas complementares.

Os segredos nunca vão para o repositório em texto simples. O CI/CD automatiza o teste e a aplicação das alterações, tendo o Git como fonte de verdade. A Aula 08 retoma esta base: a automatização que aqui permite criar e configurar máquinas a pedido é a condição para o *auto-scaling*.

Leitura Complementar

- Morris, K. *Infrastructure as Code*, 2.ª ed., O'Reilly, 2020.
- Brikman, Y. *Terraform: Up & Running*, 3.ª ed., O'Reilly, 2022.
- Hochstein, L. e Moser, R. *Ansible: Up and Running*, 3.ª ed., O'Reilly, 2022.
- Documentação oficial do Ansible (docs.ansible.com) e do Terraform (developer.hashicorp.com/terraform).

Para aprofundar a matéria, recomendam-se as obras de Morris sobre Infrastructure as Code, de Brikman sobre o Terraform e de Hochstein e Moser sobre o Ansible, todas da O'Reilly. A documentação oficial das duas ferramentas é a referência atualizada para a sintaxe e para os módulos e providers disponíveis.

Obrigada.

Questões e comentários

Este é o espaço para questões e comentários. Sugere-se que o estudante, antes da Aula 08, consiga explicar com palavras suas três coisas: a diferença entre provisionamento e gestão de configuração, o que significa dizer que uma operação é idempotente e porque é que o state file do Terraform deve ser tratado como um segredo.